Playwright with Python — Locators & Actions

1. Locators — Finding Elements

Playwright gives multiple locator strategies to target elements.
These are web-first → they wait for the element to be ready before acting.

1.1 Role-based Locators

Use ARIA roles (button, textbox, checkbox, link, heading, etc.).
Works best with accessible apps.

```
page.get_by_role("button", name="Login").click()
page.get_by_role("textbox", name="Username").fill("rupesh")
```

- Clear, semantic, and stable.
- Depends on proper ARIA roles in the app.

1.2 Text-based Locators

Locate elements by visible text or label.

```
page.get_by_text("Welcome").click()
page.get_by_label("Password").fill("mypassword")
page.get_by_placeholder("Search...").fill("Playwright")
```

- Easy to understand.
- Fragile if UI text changes frequently.

1.3 CSS Selectors

Standard way of locating by ID, class, attributes, hierarchy.

```
page.locator("button#login").click()
page.locator("input.username").fill("rupesh")
page.locator("div.menu > ul > li:first-child").hover()
```

- Flexible, powerful, fast.
- Requires knowledge of CSS selectors.

1.4 XPath Selectors

Based on XML path queries.

```
page.locator("//button[text()='Login']").click()
page.locator("//input[@id='username']").fill("rupesh")
```

- Useful when CSS selectors are hard.
- Harder to read and maintain.

1.5 Chained Locators

Scope locators inside a parent container.

```
form = page.locator("form#login-form")
form.locator("input[name='username']").fill("rupesh")
form.locator("input[name='password']").fill("secret")
form.locator("button[type='submit']").click()
```

- Keeps tests clean and avoids ambiguity.

1.6 First/Last/Nth Match

For multiple matching elements.

```
page.locator("button").first.click()
page.locator("button").last.click()
page.locator("button").nth(2).click()  # 0-based index
```

1.7 Locator Filters

Refine locators with conditions.

```
page.locator("div.item").filter(has_text="Special").click()
```

2. Actions — Interacting with Elements

After locating, Playwright provides high-level user actions.
All actions auto-wait for the element to be visible and stable.

2.1 Clicking

```
page.click("text=Login")
page.locator("#submit").click()
```

Simulates a user click.

2.2 Typing & Filling Inputs

```
page.fill("#username", "rupesh") # replaces existing text
page.type("#search", "Playwright") # types character by character
```

2.3 Keyboard Actions

```
page.press("#search", "Enter")
page.keyboard.type("Hello World")
page.keyboard.press("Control+A")
```

Useful for shortcuts or special key presses.

2.4 Mouse Actions

```
page.hover(".menu-item")
page.mouse.move(100, 200)
page.mouse.down()
page.mouse.up()
```

Fine control over mouse movements.

2.5 Checkboxes & Radio Buttons

```
page.check("#remember-me")
page.uncheck("#subscribe")
page.locator("input[type=radio][value='Male']").check()
```

Playwright ensures the element is checked/unchecked reliably.

2.6 Dropdowns (Select)

```
page.select_option("#country", "India")
page.select_option("#country", label="United States")
page.select_option("#country", index=2)
```

Can select by value, label, or index.

2.7 File Upload

```
page.set_input_files("#upload", "sample.pdf")
```

Works for `<input type="file">`.

2.8 Drag and Drop

```
page.drag_and_drop("#src", "#target")
```

Simulates drag-and-drop gesture.

2.9 Frames (iFrames)

```
frame = page.frame(name="iframe1")
frame.click("#submit")
```

Needed because iframes are separate documents.

2.10 JavaScript Dialogs (alert, confirm, prompt)

```
page.on("dialog", lambda dialog: dialog.accept())
page.click("#trigger-alert")
```

Handle dialogs programmatically.

2.11 Screenshots & Videos

```
page.screenshot(path="page.png")
page.video.save_as("recording.webm")
```

Great for debugging.

2.12 Waits

```
page.wait_for_selector("#welcome-message")
page.wait_for_timeout(2000)  # Hard wait (avoid unless debugging)
```

Playwright auto-waits, but explicit waits can be used.

Key Takeaways

Locators: Find elements using role, text, label, placeholder, css, xpath, chaining, filters.

Actions: Perform realistic user operations → click, fill, press, hover, drag, drop, check, select.

Web-First: All actions auto-wait → fewer flaky tests.